Controlled Evaluation of QML Circuit C for Network Intrusion Detection

1. Research Objective

Evaluate a four-qubit variational quantum classifier (Circuit C) for binary network intrusion detection using the UNSW-NB15 dataset.

The experiment will measure classification performance, false-positive behaviour, training cost, and sensitivity to stochastic training. Results will be compared against established classical baselines under documented evaluation conditions.

2. Research Questions

1. Can Circuit C distinguish benign network traffic from malicious traffic on an untouched held-out test set?
2. What detection performance and false-positive rate does the classifier achieve after validation-based threshold calibration?
3. How stable are results across independent training seeds?
4. How does Circuit C compare with classical machine-learning baselines in predictive performance and computational cost?

3. Experimental Hypotheses

- **H1 — Detection:** Circuit C achieves better-than-chance discrimination on held-out test data.
- **H2 — Operational trade-off:** Validation-based threshold calibration can control false positives, although the constraint may not hold on unseen test data.
- **H3 — Stability:** Performance varies across stochastic training runs; this variability will be measured rather than hidden.
- **H4 — Comparative performance:** Circuit C's performance and training cost can be compared with classical baselines, but no quantum advantage is assumed.

4. Dataset and Evaluation Protocol

- Dataset: UNSW-NB15.
- Training partition: 2,000 sampled records from the official training CSV.
- Validation partition: 1,000 records from the official test CSV.
- Final test partition: 2,000 records from the official test CSV, disjoint from validation.
- Feature representation: four PCA components mapped to rotation angles in $[0,\pi]$.
- Preprocessing, PCA, and scaling must be fitted using training data only.
- Threshold and output-polarity selection use validation data only.
- Final-test results must not influence training, model selection, or threshold selection.

The split manifest and preprocessing artifacts from the reproducibility audit are authoritative inputs to this experiment. Their integrity will be checked before training.

5. Circuit Architecture

Circuit C consists of:

1. Four-qubit angle encoding using the four PCA features.
2. A trainable (R_Y) rotation on each qubit.
3. A linear CNOT entanglement chain.
4. A trainable (R_Z) rotation on each qubit.
5. Measurement of qubit 0 to estimate the model output probability.

This produces eight trainable parameters.

6. Reporting Requirements

Report ROC-AUC, accuracy, precision, recall, F1-score, false-positive rate, confusion matrix, training duration, and run-to-run variability. Include the random seeds, optimizer settings, shot count, dependency versions, and exact split identifiers.

Historical benchmark results must remain separate from results produced by this controlled protocol.

7. Interpretation Boundary

This is a controlled evaluation of a quantum machine-learning classifier. Successful simulation or competitive classification metrics alone do not demonstrate quantum advantage. Any comparative claim must be supported by a fair, reproducible experiment.

```
# 2.1 Resolve project root and validate required artifacts

import json
from pathlib import Path

PROJECT_ROOT = Path.cwd().resolve().parents[1]

assert PROJECT_ROOT.name == "Aeges-Q", (
    f"Unexpected project root: {PROJECT_ROOT}"
)

ARTIFACT_DIR = PROJECT_ROOT / "artifacts"
QML_ARTIFACT_DIR = ARTIFACT_DIR / "qml"

SPLIT_MANIFEST_PATH = QML_ARTIFACT_DIR / "qml_split_manifest.json"
PREPROCESSOR_PATH = (
    ARTIFACT_DIR / "classical" / "variant_e_preprocessor.joblib"
)
HISTORICAL_RESULTS_PATH = (
```

```
    QML_ARTIFACT_DIR / "qml_circuit_c_local_results.csv"
)

required_paths = {
    "split manifest": SPLIT_MANIFEST_PATH,
    "classical preprocessor": PREPROCESSOR_PATH,
    "historical results": HISTORICAL_RESULTS_PATH,
}

for name, path in required_paths.items():
    assert path.is_file(), f"Missing {name}: {path}"
    print(f"[OK] {name}: {path.relative_to(PROJECT_ROOT)}")

with SPLIT_MANIFEST_PATH.open("r", encoding="utf-8") as f:
    split_manifest = json.load(f)

print("\nProject root:", PROJECT_ROOT)
print("Manifest top-level keys:", list(split_manifest.keys()))
```

```
[OK] split manifest: artifacts\qml\qml_split_manifest.json
[OK] classical preprocessor: artifacts\classical\
variant_e_preprocessor.joblib
[OK] historical results: artifacts\qml\qml_circuit_c_local_results.csv

Project root: C:\Projects\Aeges-Q
Manifest top-level keys: ['random_state', 'train_csv_rows',
'validation_csv_rows', 'test_csv_rows']
```

```
# 2.2 Validate split manifest integrity

import numpy as np

expected_sizes = {
    "train_csv_rows": 2_000,
    "validation_csv_rows": 1_000,
    "test_csv_rows": 2_000,
}

for key, expected_size in expected_sizes.items():
    assert key in split_manifest, f"Missing manifest key: {key}"

    indices = np.asarray(split_manifest[key], dtype=np.int64)

    assert indices.ndim == 1, f"{key} must be a one-dimensional array"
    assert len(indices) == expected_size, (
        f"{key}: expected {expected_size} rows, found {len(indices)}"
    )
    assert len(np.unique(indices)) == len(indices), (
        f"Duplicate row indices found in {key}"
    )
```

```
    split_manifest[key] = indices
    print(f"[OK] {key}: {len(indices):,} unique rows")

train_rows = set(split_manifest["train_csv_rows"].tolist())
validation_rows = set(split_manifest["validation_csv_rows"].tolist())
test_rows = set(split_manifest["test_csv_rows"].tolist())

# Validation and final test both refer to rows in the official testing
CSV.
assert validation_rows.isdisjoint(test_rows), (
    "Validation and final test overlap!"
)

# Training indices refer to a different CSV, so their numeric values
# must NOT be compared with test CSV row indices.
print("\n[OK] Validation and final test are disjoint.")
print("[INFO] Training indices belong to the separate official
training CSV.")
print("[INFO] Training and test indices are validated independently.")
```

```
[OK] train_csv_rows: 2,000 unique rows
[OK] validation_csv_rows: 1,000 unique rows
[OK] test_csv_rows: 2,000 unique rows

[OK] Validation and final test are disjoint.
[INFO] Training indices belong to the separate official training CSV.
[INFO] Training and test indices are validated independently.
```

```
# 2.3 Load official data and validate selected records

import pandas as pd
import numpy as np
import joblib

TRAIN_CSV = PROJECT_ROOT / "data" / "raw" / "UNSW_NB15_training-
set.csv"
TEST_CSV = PROJECT_ROOT / "data" / "raw" / "UNSW_NB15_testing-set.csv"

for path in (TRAIN_CSV, TEST_CSV):
    assert path.is_file(), f"Dataset file not found: {path}"

train_df = pd.read_csv(TRAIN_CSV)
official_test_df = pd.read_csv(TEST_CSV)

assert len(train_df) == 82_332, (
    f"Unexpected training CSV size: {len(train_df):,}"
)
assert len(official_test_df) == 175_341, (
    f"Unexpected testing CSV size: {len(official_test_df):,}"
)
assert "label" in train_df.columns
```

```python
assert "label" in official_test_df.columns

train_indices = split_manifest["train_csv_rows"]
validation_indices = split_manifest["validation_csv_rows"]
test_indices = split_manifest["test_csv_rows"]

assert train_indices.min() >= 0 and train_indices.max() <
len(train_df)
assert validation_indices.min() >= 0 and validation_indices.max() <
len(official_test_df)
assert test_indices.min() >= 0 and test_indices.max() <
len(official_test_df)

# Positional selection: manifest values are CSV row positions.
train_subset = train_df.iloc[train_indices].copy()
validation_subset = official_test_df.iloc[validation_indices].copy()
test_subset = official_test_df.iloc[test_indices].copy()

assert len(train_subset) == 2_000
assert len(validation_subset) == 1_000
assert len(test_subset) == 2_000

for name, frame in [
    ("Train", train_subset),
    ("Validation", validation_subset),
    ("Final test", test_subset),
]:
    assert frame["label"].isin([0, 1]).all(), (
        f"{name} contains unexpected target values"
    )
    print(
        f"{name:10s} | rows={len(frame):,} "
        f"| benign={(frame['label'] == 0).sum():,} "
        f"| attack={(frame['label'] == 1).sum():,}"
    )

preprocessor = joblib.load(PREPROCESSOR_PATH)

print("\n[OK] Official CSVs loaded and manifest indices applied.")
print("Raw columns:", len(train_df.columns))
print("Preprocessor type:", type(preprocessor).__name__)
print("Preprocessor artifact:",
PREPROCESSOR_PATH.relative_to(PROJECT_ROOT))
```

```
Train      | rows=2,000 | benign=899 | attack=1,101
Validation | rows=1,000 | benign=319 | attack=681
Final test | rows=2,000 | benign=639 | attack=1,361

[OK] Official CSVs loaded and manifest indices applied.
Raw columns: 45
```

```
Preprocessor type: ColumnTransformer
Preprocessor artifact: artifacts\classical\
variant_e_preprocessor.joblib

# 2.4 Training-only feature reduction and angle encoding

from sklearn.decomposition import PCA
from sklearn.preprocessing import MinMaxScaler

TARGET_COLUMN = "label"
DROP_COLUMNS = [TARGET_COLUMN]

X_train_raw = train_subset.drop(columns=DROP_COLUMNS)
X_validation_raw = validation_subset.drop(columns=DROP_COLUMNS)
X_test_raw = test_subset.drop(columns=DROP_COLUMNS)

y_train = train_subset[TARGET_COLUMN].to_numpy(dtype=np.int64)
y_validation =
validation_subset[TARGET_COLUMN].to_numpy(dtype=np.int64)
y_test = test_subset[TARGET_COLUMN].to_numpy(dtype=np.int64)

# The saved ColumnTransformer must transform each split consistently.
X_train_transformed = preprocessor.transform(X_train_raw)
X_validation_transformed = preprocessor.transform(X_validation_raw)
X_test_transformed = preprocessor.transform(X_test_raw)

# Sparse outputs are converted only if necessary.
if hasattr(X_train_transformed, "toarray"):
    X_train_transformed = X_train_transformed.toarray()
if hasattr(X_validation_transformed, "toarray"):
    X_validation_transformed = X_validation_transformed.toarray()
if hasattr(X_test_transformed, "toarray"):
    X_test_transformed = X_test_transformed.toarray()

X_train_transformed = np.asarray(X_train_transformed,
dtype=np.float64)
X_validation_transformed = np.asarray(X_validation_transformed,
dtype=np.float64)
X_test_transformed = np.asarray(X_test_transformed, dtype=np.float64)

assert X_train_transformed.ndim == 2
assert X_train_transformed.shape[1] == 177, (
    f"Expected 177 transformed features; got
{X_train_transformed.shape[1]}"
)
assert X_validation_transformed.shape[1] ==
X_train_transformed.shape[1]
assert X_test_transformed.shape[1] == X_train_transformed.shape[1]

assert np.isfinite(X_train_transformed).all()
```

```
assert np.isfinite(X_validation_transformed).all()
assert np.isfinite(X_test_transformed).all()

pca = PCA(n_components=4, random_state=20261012)
X_train_pca = pca.fit_transform(X_train_transformed)
X_validation_pca = pca.transform(X_validation_transformed)
X_test_pca = pca.transform(X_test_transformed)

angle_scaler = MinMaxScaler(feature_range=(0, np.pi))
X_train_angles = angle_scaler.fit_transform(X_train_pca)
X_validation_angles = angle_scaler.transform(X_validation_pca)
X_test_angles = angle_scaler.transform(X_test_pca)

print("Transformed feature shapes:")
print("  Train:      ", X_train_transformed.shape)
print("  Validation:", X_validation_transformed.shape)
print("  Final test:", X_test_transformed.shape)

print("\nAngle-array shapes:")
print("  Train:      ", X_train_angles.shape)
print("  Validation:", X_validation_angles.shape)
print("  Final test:", X_test_angles.shape)

print("\nExplained variance ratio:", pca.explained_variance_ratio_)
print(f"Total variance explained:
{pca.explained_variance_ratio_.sum():.4%}")

assert X_train_angles.shape == (2_000, 4)
assert X_validation_angles.shape == (1_000, 4)
assert X_test_angles.shape == (2_000, 4)
assert np.isfinite(X_train_angles).all()
assert np.isfinite(X_validation_angles).all()
assert np.isfinite(X_test_angles).all()

print("\n[OK] Feature transformation and training-only PCA/scaling
verified.")
```

```
Transformed feature shapes:
  Train:      (2000, 177)
  Validation: (1000, 177)
  Final test: (2000, 177)

Angle-array shapes:
  Train:      (2000, 4)
  Validation: (1000, 4)
  Final test: (2000, 4)

Explained variance ratio: [0.23608706 0.13531218 0.09190311
0.05781536]
Total variance explained: 52.1118%
```

```
[OK] Feature transformation and training-only PCA/scaling verified.
```

3. Circuit C Implementation

Circuit C uses four PCA-derived input angles, four trainable (R_Y) rotations, a linear CNOT chain, and four trainable (R_Z) rotations. Measurement of qubit 0 provides the estimated output probability.

The implementation will be validated on individual samples before optimization begins. The simulator seed and shot count will be fixed for the initial controlled run. Stochastic training will be addressed explicitly in subsequent experiments.

```
# 3.1 Define Circuit C and probability inference

import time
import numpy as np

from qiskit import QuantumCircuit, transpile
from qiskit.circuit import ParameterVector
from qiskit_aer import AerSimulator

N_QUBITS = 4
N_FEATURES = 4
N_TRAINABLE_PARAMS = 8
SHOTS = 256
SIMULATOR_SEED = 20261012

feature_params = ParameterVector("x", N_FEATURES)
theta_params = ParameterVector("theta", N_TRAINABLE_PARAMS)

circuit_c = QuantumCircuit(N_QUBITS, 1)

# Input feature encoding
for q in range(N_QUBITS):
    circuit_c.ry(feature_params[q], q)

# Trainable RY layer
for q in range(N_QUBITS):
    circuit_c.ry(theta_params[q], q)

# Linear entanglement chain
for q in range(N_QUBITS - 1):
    circuit_c.cx(q, q + 1)

# Trainable RZ layer
for q in range(N_QUBITS):
    circuit_c.rz(theta_params[N_QUBITS + q], q)

# Measure qubit 0
```

```
circuit_c.measure(0, 0)

simulator = AerSimulator()
compiled_circuit_c = transpile(
    circuit_c,
    simulator,
    optimization_level=1,
)

assert circuit_c.num_qubits == 4
assert len(circuit_c.parameters) == 12

PARAMETER_ORDER = list(feature_params) + list(theta_params)

def circuit_c_probability(
    angles,
    theta,
    shots=SHOTS,
    seed_simulator=SIMULATOR_SEED,
):
    """Estimate P(measured qubit 0 = 1) for one four-feature
sample."""
    angles = np.asarray(angles, dtype=np.float64)
    theta = np.asarray(theta, dtype=np.float64)

    if angles.shape != (N_FEATURES,):
        raise ValueError(f"Expected {N_FEATURES} input angles; got
{angles.shape}")
    if theta.shape != (N_TRAINABLE_PARAMS,):
        raise ValueError(
            f"Expected {N_TRAINABLE_PARAMS} trainable parameters; got
{theta.shape}"
        )
    if not np.isfinite(angles).all() or not np.isfinite(theta).all():
        raise ValueError("Angles and parameters must be finite")
    if shots < 1:
        raise ValueError("shots must be positive")

    values = np.concatenate([angles, theta])
    bound = compiled_circuit_c.assign_parameters(
        dict(zip(PARAMETER_ORDER, values)),
        inplace=False,
    )

    result = simulator.run(
        bound,
        shots=shots,
        seed_simulator=seed_simulator,
    ).result()
```

```
    counts = result.get_counts()
    return counts.get("1", 0) / shots


# Deterministic smoke test using fixed parameters and simulator seed
theta_smoke = np.zeros(N_TRAINABLE_PARAMS, dtype=np.float64)

p1 = circuit_c_probability(
    X_train_angles[0],
    theta_smoke,
    shots=SHOTS,
    seed_simulator=SIMULATOR_SEED,
)
p2 = circuit_c_probability(
    X_train_angles[0],
    theta_smoke,
    shots=SHOTS,
    seed_simulator=SIMULATOR_SEED,
)

assert 0.0 <= p1 <= 1.0
assert p1 == p2, "Fixed-seed inference is not reproducible"

print("[OK] Circuit C built.")
print("Qubits:", circuit_c.num_qubits)
print("Trainable parameters:", N_TRAINABLE_PARAMS)
print("Measurement:", "qubit 0")
print("Smoke-test probability:", p1)
print("[OK] Repeated inference with the same simulator seed matches.")
```

```
[OK] Circuit C built.
Qubits: 4
Trainable parameters: 8
Measurement: qubit 0
Smoke-test probability: 0.6015625
[OK] Repeated inference with the same simulator seed matches.
```

3.2 Training Objective and Optimizer

The model estimates (p(y=1\mid x)) from the frequency of measuring qubit 0 in state 1.

Binary cross-entropy is used as the training objective. The Simultaneous Perturbation Stochastic Approximation (SPSA) optimizer estimates the gradient using two objective evaluations per update, making it suitable for a small parameterized quantum circuit.

The initial experiment uses a fixed training order and explicit random seeds. The optimizer configuration and shot count will be recorded with the results.

```
# 3.2 Binary cross-entropy and SPSA optimizer
```

```
import numpy as np

TRAINING_SEED = 20261012
SPSA_A = 0.10
SPSA_C = 0.08
SPSA_ALPHA = 0.602
SPSA_GAMMA = 0.101
BCE_EPSILON = 1e-7


def binary_cross_entropy(y_true, probabilities, epsilon=BCE_EPSILON):
    """Mean binary cross-entropy with numerical clipping."""
    y_true = np.asarray(y_true, dtype=np.float64)
    probabilities = np.asarray(probabilities, dtype=np.float64)

    if y_true.shape != probabilities.shape:
        raise ValueError("Targets and probabilities must have
identical shapes")
    if not np.isfinite(probabilities).all():
        raise ValueError("Probabilities contain non-finite values")
    if not np.isin(y_true, [0, 1]).all():
        raise ValueError("Targets must be binary")

    probabilities = np.clip(probabilities, epsilon, 1.0 - epsilon)

    return float(
        -np.mean(
            y_true * np.log(probabilities)
            + (1.0 - y_true) * np.log(1.0 - probabilities)
        )
    )


def predict_probabilities(
    X_angles,
    theta,
    shots=SHOTS,
    seed_base=SIMULATOR_SEED,
):
    """Return one measured probability per sample."""
    X_angles = np.asarray(X_angles, dtype=np.float64)

    if X_angles.ndim != 2 or X_angles.shape[1] != N_FEATURES:
        raise ValueError(f"Expected an (n, {N_FEATURES}) input array")

    probabilities = np.empty(len(X_angles), dtype=np.float64)

    for i, angles in enumerate(X_angles):
        probabilities[i] = circuit_c_probability(
            angles,
```

```
            theta,
            shots=shots,
            seed_simulator=seed_base + i,
        )

    return probabilities


def spsa_step(
    theta,
    objective,
    iteration,
    rng,
    config=EXPERIMENT_CONFIG,
):
    """Perform one SPSA update using the frozen experiment
configuration."""
    theta = np.asarray(theta, dtype=np.float64)

    ak = config.spsa_a / ((iteration + 1) ** config.spsa_alpha)
    ck = config.spsa_c / ((iteration + 1) ** config.spsa_gamma)

    delta = rng.choice([-1.0, 1.0], size=theta.shape)

    loss_plus = objective(theta + ck * delta)
    loss_minus = objective(theta - ck * delta)

    gradient_estimate = (
        (loss_plus - loss_minus) / (2.0 * ck)
    ) * delta

    theta_updated = theta - ak * gradient_estimate

    if not np.isfinite(theta_updated).all():
        raise FloatingPointError("SPSA produced non-finite
parameters")

    return theta_updated, {
        "loss_plus": float(loss_plus),
        "loss_minus": float(loss_minus),
        "gradient_norm": float(np.linalg.norm(gradient_estimate)),
        "learning_rate": float(ak),
        "perturbation": float(ck),
    }


# Unit checks for the loss function
assert np.isfinite(binary_cross_entropy([0, 1], [0.2, 0.8]))
assert binary_cross_entropy([0, 1], [0.1, 0.9]) <
binary_cross_entropy(
```

```
    [0, 1], [0.9, 0.1]
)

# One-step optimizer test on a fixed, tiny batch.
# This validates execution only; it is not a model-performance result.
rng_smoke = np.random.default_rng(TRAINING_SEED)
theta_initial = rng_smoke.uniform(
    -np.pi, np.pi, size=N_TRAINABLE_PARAMS
)

X_smoke = X_train_angles[:4]
y_smoke = y_train[:4]

def smoke_objective(theta):
    p = predict_probabilities(
        X_smoke,
        theta,
        shots=SHOTS,
        seed_base=SIMULATOR_SEED,
    )
    return binary_cross_entropy(y_smoke, p)

theta_updated, diagnostics = spsa_step(
    theta_initial,
    smoke_objective,
    iteration=0,
    rng=rng_smoke,
)

assert theta_updated.shape == (N_TRAINABLE_PARAMS,)
assert np.isfinite(theta_updated).all()
assert np.isfinite(list(diagnostics.values())).all()

print("[OK] BCE checks passed.")
print("[OK] One SPSA update executed.")
print("Initial parameters:", np.round(theta_initial, 4))
print("Updated parameters:", np.round(theta_updated, 4))
print("Diagnostics:", diagnostics)
```

```
[OK] BCE checks passed.
[OK] One SPSA update executed.
Initial parameters: [ 0.6948 -0.5497  2.4625  0.9556  0.6998  1.6093 -
0.4852  3.1047]
Updated parameters: [ 0.487  -0.3419  2.6702  1.1634  0.9076  1.4015 -
0.2774  3.3125]
Diagnostics: {'loss_plus': 2.040513578786614, 'loss_minus':
1.7080931024729291, 'gradient_norm': 5.876419325166716,
'learning_rate': 0.1, 'perturbation': 0.08}
```

3.3 Training Configuration

The first controlled run will use a fixed dataset split and an explicitly recorded optimizer configuration. Training duration and objective evaluations will be recorded.

The run is a protocol-validation experiment, not a claim of optimal performance. Hyperparameters will not be selected using final-test results. Any later seed repetitions will use the same data partitions and report variability rather than selecting the best-performing run.

```
# 3.3 Freeze the first-run configuration

from dataclasses import dataclass, asdict
import json
from pathlib import Path

@dataclass(frozen=True)
class CircuitCConfig:
    experiment_id: str = "qml_circuit_c_controlled_v1"
    training_seed: int = 20261012
    simulator_seed: int = 20261012
    shots: int = 256
    train_samples: int = 2000
    validation_samples: int = 1000
    test_samples: int = 2000
    spsa_iterations: int = 20
    spsa_a: float = 0.10
    spsa_c: float = 0.08
    spsa_alpha: float = 0.602
    spsa_gamma: float = 0.101
    threshold_objective: str =
"maximize_validation_f1_subject_to_fpr_le_0.10"

EXPERIMENT_CONFIG = CircuitCConfig()

assert len(X_train_angles) == EXPERIMENT_CONFIG.train_samples
assert len(X_validation_angles) ==
EXPERIMENT_CONFIG.validation_samples
assert len(X_test_angles) == EXPERIMENT_CONFIG.test_samples
assert SHOTS == EXPERIMENT_CONFIG.shots

config_dict = asdict(EXPERIMENT_CONFIG)

print(json.dumps(config_dict, indent=2))
print("\nCircuit executions for training objective evaluations:")
print(
    f"  {EXPERIMENT_CONFIG.spsa_iterations} iterations "
    f"× 2 SPSA evaluations "
    f"× {EXPERIMENT_CONFIG.train_samples:,} samples "
    f"= {2 * EXPERIMENT_CONFIG.spsa_iterations *
EXPERIMENT_CONFIG.train_samples:,}"
```

```
)
print(
    "\nNote: this excludes validation, final-test inference, "
    "and any additional evaluation."
)
```

```
{
  "experiment_id": "qml_circuit_c_controlled_v1",
  "training_seed": 20261012,
  "simulator_seed": 20261012,
  "shots": 256,
  "train_samples": 2000,
  "validation_samples": 1000,
  "test_samples": 2000,
  "spsa_iterations": 20,
  "spsa_a": 0.1,
  "spsa_c": 0.08,
  "spsa_alpha": 0.602,
  "spsa_gamma": 0.101,
  "threshold_objective":
"maximize_validation_f1_subject_to_fpr_le_0.10"
}

Circuit executions for training objective evaluations:
  20 iterations × 2 SPSA evaluations × 2,000 samples = 80,000

Note: this excludes validation, final-test inference, and any
additional evaluation.
```

3.4 Circuit C Training

Train Circuit C using the frozen SPSA configuration. Each iteration records the positive and negative perturbation losses, gradient estimate norm, updated parameters, and elapsed time.

The training artifact is saved after each iteration. This permits recovery from interruption and preserves the full optimization trace. Validation and final-test data are excluded from the training objective.

Runtime Pilot

Measure the wall-clock cost of one SPSA iteration on a fixed subset of 128 training records. This pilot is used only to estimate computational cost; its parameters and metrics are not used as the final model result.

The full experiment configuration remains unchanged. Any decision to alter the training budget must be recorded explicitly before the main run.

```
# 3.4.1 Runtime pilot — 128 samples, one SPSA iteration

import time
```

```
import numpy as np

PILOT_SIZE = 128
PILOT_SEED = 20261013

pilot_rng = np.random.default_rng(PILOT_SEED)
pilot_indices = pilot_rng.choice(
    len(X_train_angles),
    size=PILOT_SIZE,
    replace=False,
)

X_pilot = X_train_angles[pilot_indices]
y_pilot = y_train[pilot_indices]

theta_pilot = np.random.default_rng(TRAINING_SEED).uniform(
    -np.pi, np.pi, size=N_TRAINABLE_PARAMS
)

def pilot_objective(theta):
    probabilities = predict_probabilities(
        X_pilot,
        theta,
        shots=EXPERIMENT_CONFIG.shots,
        seed_base=EXPERIMENT_CONFIG.simulator_seed,
    )
    return binary_cross_entropy(y_pilot, probabilities)

pilot_optimizer_rng = np.random.default_rng(PILOT_SEED)

pilot_start = time.perf_counter()

theta_pilot_updated, pilot_diagnostics = spsa_step(
    theta_pilot,
    pilot_objective,
    iteration=0,
    rng=pilot_optimizer_rng,
    config=EXPERIMENT_CONFIG,
)

pilot_elapsed_seconds = time.perf_counter() - pilot_start

estimated_seconds_per_iteration = (
    pilot_elapsed_seconds
    * EXPERIMENT_CONFIG.train_samples
    / PILOT_SIZE
)

print(f"Pilot samples: {PILOT_SIZE}")
print(f"Pilot iteration duration: {pilot_elapsed_seconds:.2f}
```

```
seconds")
print(
    "Estimated full-data iteration: "
    f"{estimated_seconds_per_iteration / 60:.2f} minutes"
)
print(
    "Estimated 20-iteration training duration: "
    f"{estimated_seconds_per_iteration *
EXPERIMENT_CONFIG.spsa_iterations / 3600:.2f} hours"
)
print("Pilot diagnostics:", pilot_diagnostics)

assert np.isfinite(theta_pilot_updated).all()
assert np.isfinite(pilot_elapsed_seconds)
```

```
Pilot samples: 128
Pilot iteration duration: 1.54 seconds
Estimated full-data iteration: 0.40 minutes
Estimated 20-iteration training duration: 0.13 hours
Pilot diagnostics: {'loss_plus': 1.4032578272232072, 'loss_minus':
1.4977124769741907, 'gradient_norm': 1.669738083838017,
'learning_rate': 0.1, 'perturbation': 0.08}
```

Full Training Run

Train Circuit C on the complete 2,000-sample training subset using the frozen 20-iteration SPSA configuration. The pilot is excluded from the training history.

Save the parameter vector and optimization history after every iteration. If execution is interrupted, the latest completed checkpoint remains available. Validation and final-test records are not used in the training objective.

```
# 3.4.2 Full Circuit C training with per-iteration checkpoints

import json
import time
import numpy as np
import pandas as pd
from pathlib import Path

RUN_DIR = QML_ARTIFACT_DIR / EXPERIMENT_CONFIG.experiment_id
RUN_DIR.mkdir(parents=True, exist_ok=True)

CHECKPOINT_PATH = RUN_DIR / "training_checkpoint.npz"
HISTORY_PATH = RUN_DIR / "training_history.csv"
CONFIG_PATH = RUN_DIR / "experiment_config.json"

# Preserve the exact configuration used for this run.
with CONFIG_PATH.open("w", encoding="utf-8") as f:
    json.dump(config_dict, f, indent=2)
```

```
# Do not silently overwrite an existing run.
if CHECKPOINT_PATH.exists() or HISTORY_PATH.exists():
    raise FileExistsError(
        f"Training artifacts already exist in {RUN_DIR}. "
        "Inspect them before restarting or choose a new
experiment_id."
    )

training_rng = np.random.default_rng(EXPERIMENT_CONFIG.training_seed)
theta = training_rng.uniform(-np.pi, np.pi, size=N_TRAINABLE_PARAMS)

training_history = []
training_start = time.perf_counter()

def training_objective(parameters):
    probabilities = predict_probabilities(
        X_train_angles,
        parameters,
        shots=EXPERIMENT_CONFIG.shots,
        seed_base=EXPERIMENT_CONFIG.simulator_seed,
    )
    return binary_cross_entropy(y_train, probabilities)

for iteration in range(EXPERIMENT_CONFIG.spsa_iterations):
    iteration_start = time.perf_counter()

    theta, diagnostics = spsa_step(
        theta,
        training_objective,
        iteration=iteration,
        rng=training_rng,
        config=EXPERIMENT_CONFIG,
    )

    iteration_elapsed = time.perf_counter() - iteration_start
    total_elapsed = time.perf_counter() - training_start

    row = {
        "iteration": iteration + 1,
        "loss_plus": diagnostics["loss_plus"],
        "loss_minus": diagnostics["loss_minus"],
        "gradient_norm": diagnostics["gradient_norm"],
        "learning_rate": diagnostics["learning_rate"],
        "perturbation": diagnostics["perturbation"],
        "iteration_seconds": iteration_elapsed,
        "elapsed_seconds": total_elapsed,
    }
    training_history.append(row)
```

```
    # Save the latest completed parameters and full history.
    np.savez(
        CHECKPOINT_PATH,
        theta=theta,
        completed_iterations=iteration + 1,
        training_seed=EXPERIMENT_CONFIG.training_seed,
        simulator_seed=EXPERIMENT_CONFIG.simulator_seed,
    )
    pd.DataFrame(training_history).to_csv(HISTORY_PATH, index=False)

    print(
        f"Iteration {iteration + 1:02d}/"
        f"{EXPERIMENT_CONFIG.spsa_iterations} | "
        f"L+={row['loss_plus']:.4f} | "
        f"L-={row['loss_minus']:.4f} | "
        f"|g|={row['gradient_norm']:.4f} | "
        f"{iteration_elapsed:.1f}s"
    )

training_duration = time.perf_counter() - training_start

FINAL_PARAMETERS_PATH = RUN_DIR / "circuit_c_parameters.npy"
np.save(FINAL_PARAMETERS_PATH, theta)

print(f"\nTraining completed in {training_duration / 60:.2f}
minutes.")
print("Checkpoint:", CHECKPOINT_PATH)
print("History:", HISTORY_PATH)
print("Final parameters:", FINAL_PARAMETERS_PATH)
```

```
Iteration 01/20 | L+=1.5200 | L-=1.2813 | |g|=4.2203 | 36.8s
Iteration 02/20 | L+=1.1765 | L-=1.2813 | |g|=1.9858 | 22.2s
Iteration 03/20 | L+=1.2504 | L-=1.1439 | |g|=2.1052 | 19.1s
Iteration 04/20 | L+=1.1283 | L-=1.2130 | |g|=1.7220 | 18.9s
Iteration 05/20 | L+=1.1193 | L-=1.1930 | |g|=1.5332 | 19.0s
Iteration 06/20 | L+=1.1809 | L-=1.1234 | |g|=1.2189 | 18.8s
Iteration 07/20 | L+=1.1076 | L-=1.1668 | |g|=1.2726 | 20.8s
Iteration 08/20 | L+=1.1586 | L-=1.1042 | |g|=1.1864 | 19.9s
Iteration 09/20 | L+=1.1531 | L-=1.1016 | |g|=1.1366 | 18.8s
Iteration 10/20 | L+=1.0999 | L-=1.1432 | |g|=0.9665 | 19.1s
Iteration 11/20 | L+=1.1394 | L-=1.0973 | |g|=0.9485 | 19.4s
Iteration 12/20 | L+=1.0958 | L-=1.1363 | |g|=0.9216 | 21.0s
Iteration 13/20 | L+=1.1328 | L-=1.0943 | |g|=0.8807 | 20.2s
Iteration 14/20 | L+=1.1298 | L-=1.0877 | |g|=0.9724 | 20.1s
Iteration 15/20 | L+=1.0866 | L-=1.1266 | |g|=0.9293 | 20.2s
Iteration 16/20 | L+=1.1240 | L-=1.0869 | |g|=0.8668 | 20.3s
Iteration 17/20 | L+=1.1211 | L-=1.0861 | |g|=0.8248 | 20.6s
Iteration 18/20 | L+=1.0845 | L-=1.1244 | |g|=0.9431 | 20.3s
Iteration 19/20 | L+=1.1277 | L-=1.0835 | |g|=1.0502 | 20.2s
Iteration 20/20 | L+=1.1265 | L-=1.0835 | |g|=1.0292 | 20.2s
```

```
Training completed in 6.94 minutes.
Checkpoint: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_controlled_v1\training_checkpoint.npz
History: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_controlled_v1\training_history.csv
Final parameters: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_controlled_v1\circuit_c_parameters.npy
```

4. Validation-Based Calibration

Objective

Determine the output polarity and decision threshold using validation data only.

Procedure

1. Generate validation probabilities using the trained Circuit C parameters.
2. Evaluate both the original and inverted probability orientations.
3. Select the threshold that maximizes validation F1 subject to a false-positive rate of at most 10%.
4. Record the selected polarity, threshold, validation metrics, and inference duration.

The final test set will not be used for polarity selection or threshold optimization. The validation constraint is a selection criterion, not a guarantee of the same false-positive rate on unseen data.

```python
# 4.1 Generate validation probabilities

import time
import numpy as np
import pandas as pd
from pathlib import Path

PARAMETERS_PATH = RUN_DIR / "circuit_c_parameters.npy"

theta_final = np.load(PARAMETERS_PATH)

assert theta_final.shape == (N_TRAINABLE_PARAMS,)
assert np.isfinite(theta_final).all()

validation_start = time.perf_counter()

validation_probabilities = predict_probabilities(
    X_validation_angles,
    theta_final,
    shots=EXPERIMENT_CONFIG.shots,
    seed_base=EXPERIMENT_CONFIG.simulator_seed + 100_000,
)
```

```
validation_duration = time.perf_counter() - validation_start

assert validation_probabilities.shape == (len(y_validation),)
assert np.isfinite(validation_probabilities).all()
assert np.all((validation_probabilities >= 0) &
(validation_probabilities <= 1))

VALIDATION_PROBABILITIES_PATH = RUN_DIR /
"validation_probabilities.npy"
np.save(VALIDATION_PROBABILITIES_PATH, validation_probabilities)

print("[OK] Validation inference completed.")
print(f"Samples: {len(validation_probabilities):,}")
print(f"Duration: {validation_duration:.2f} seconds")
print(f"Mean probability: {validation_probabilities.mean():.4f}")
print(f"Std probability:  {validation_probabilities.std():.4f}")
print(f"Unique probability values:
{len(np.unique(validation_probabilities))}")
print("Saved:", VALIDATION_PROBABILITIES_PATH)
```

```
[OK] Validation inference completed.
Samples: 1,000
Duration: 5.28 seconds
Mean probability: 0.5696
Std probability:  0.2378
Unique probability values: 187
Saved: C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_controlled_v1\
validation_probabilities.npy
```

4.2 Threshold Selection

Evaluate both probability orientations: (p) and (1-p). For each orientation, select the threshold that maximizes validation F1 while satisfying the constraint (FPR \leq 0.10).

The threshold search is performed exclusively on validation predictions. If no threshold satisfies the constraint, the calibration must report that failure rather than silently relaxing it.

```
# 4.2 Validation-only polarity and threshold calibration

import numpy as np
import pandas as pd

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
```

```
)

MAX_VALIDATION_FPR = 0.10


def calculate_binary_metrics(y_true, y_pred, probabilities):
    tn, fp, fn, tp = confusion_matrix(
        y_true, y_pred, labels=[0, 1]
    ).ravel()

    return {
        "accuracy": accuracy_score(y_true, y_pred),
        "precision": precision_score(
            y_true, y_pred, zero_division=0
        ),
        "recall": recall_score(
            y_true, y_pred, zero_division=0
        ),
        "f1": f1_score(y_true, y_pred, zero_division=0),
        "fpr": fp / (fp + tn) if (fp + tn) else 0.0,
        "roc_auc": roc_auc_score(y_true, probabilities),
        "tn": int(tn),
        "fp": int(fp),
        "fn": int(fn),
        "tp": int(tp),
    }


def calibrate_threshold(y_true, raw_probabilities, max_fpr=0.10):
    candidates = []

    for polarity in ("normal", "inverted"):
        probabilities = (
            raw_probabilities
            if polarity == "normal"
            else 1.0 - raw_probabilities
        )

        # Include thresholds above and below the observed range.
        thresholds = np.r_[
            np.nextafter(probabilities.max(), np.inf),
            np.unique(probabilities)[::-1],
            np.nextafter(probabilities.min(), -np.inf),
        ]

        for threshold in thresholds:
            predictions = (probabilities >=
threshold).astype(np.int64)
            metrics = calculate_binary_metrics(
                y_true, predictions, probabilities
```

```
            )

            if metrics["fpr"] <= max_fpr:
                candidates.append({
                    "polarity": polarity,
                    "threshold": float(threshold),
                    **metrics,
                })

    if not candidates:
        raise RuntimeError(
            "No candidate threshold satisfies the validation FPR
constraint."
        )

    # Deterministic tie-breaking: highest F1, then precision, then
recall.
    candidates.sort(
        key=lambda r: (
            r["f1"],
            r["precision"],
            r["recall"],
            -r["threshold"],
        ),
        reverse=True,
    )

    return candidates[0], pd.DataFrame(candidates)


calibration, feasible_thresholds = calibrate_threshold(
    y_validation,
    validation_probabilities,
    max_fpr=MAX_VALIDATION_FPR,
)

CALIBRATION_PATH = RUN_DIR / "validation_calibration.json"

import json
with CALIBRATION_PATH.open("w", encoding="utf-8") as f:
    json.dump(calibration, f, indent=2)

print("Selected polarity:", calibration["polarity"])
print(f"Selected threshold: {calibration['threshold']:.6f}")
print(f"Validation FPR:     {calibration['fpr']:.4f}")
print(f"Validation F1:      {calibration['f1']:.4f}")
print(f"Validation ROC-AUC: {calibration['roc_auc']:.4f}")
print(f"Feasible thresholds: {len(feasible_thresholds):,}")
print("\nFull validation metrics:")
for metric in (
```

```
    "accuracy", "precision", "recall", "f1", "fpr", "roc_auc",
    "tn", "fp", "fn", "tp",
):
    print(f"  {metric:10s}: {calibration[metric]}")

print("\nSaved calibration:", CALIBRATION_PATH)
```

```
Selected polarity: inverted
Selected threshold: 0.496094
Validation FPR:     0.0909
Validation F1:      0.7706
Validation ROC-AUC: 0.7361
Feasible thresholds: 115

Full validation metrics:
  accuracy  : 0.735
  precision : 0.9388185654008439
  recall    : 0.6534508076358296
  f1        : 0.7705627705627706
  fpr       : 0.09090909090909091
  roc_auc   : 0.7360648870598743
  tn        : 290
  fp        : 29
  fn        : 236
  tp        : 445

Saved calibration: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_controlled_v1\validation_calibration.json
```

5. Final Held-Out Test Evaluation

Evaluate the trained Circuit C model once on the disjoint final test partition using the polarity and threshold selected in Section 4.

No model training, polarity selection, threshold tuning, or hyperparameter selection is permitted using final-test labels. Report all predefined classification metrics and the confusion matrix, regardless of whether the results meet expectations.

The validation FPR constraint is not assumed to transfer to the final test set. The observed test FPR will be reported without post-hoc adjustment.

```
# 5.1 Evaluate once on the final held-out test set

import json
import time
import numpy as np
import pandas as pd

with CALIBRATION_PATH.open("r", encoding="utf-8") as f:
    frozen_calibration = json.load(f)
```

```
assert frozen_calibration["polarity"] in {"normal", "inverted"}
assert 0.0 <= frozen_calibration["threshold"] <= 1.0

test_start = time.perf_counter()

test_raw_probabilities = predict_probabilities(
    X_test_angles,
    theta_final,
    shots=EXPERIMENT_CONFIG.shots,
    seed_base=EXPERIMENT_CONFIG.simulator_seed + 200_000,
)

test_inference_seconds = time.perf_counter() - test_start

if frozen_calibration["polarity"] == "inverted":
    test_probabilities = 1.0 - test_raw_probabilities
else:
    test_probabilities = test_raw_probabilities.copy()

test_threshold = float(frozen_calibration["threshold"])
y_test_pred = (test_probabilities >= test_threshold).astype(np.int64)

test_metrics = calculate_binary_metrics(
    y_test,
    y_test_pred,
    test_probabilities,
)

test_metrics.update({
    "experiment_id": EXPERIMENT_CONFIG.experiment_id,
    "polarity": frozen_calibration["polarity"],
    "threshold": test_threshold,
    "inference_seconds": test_inference_seconds,
    "samples_per_second": len(y_test) / test_inference_seconds,
})

TEST_PROBABILITIES_PATH = RUN_DIR / "test_probabilities.npy"
TEST_RESULTS_PATH = RUN_DIR / "final_test_metrics.json"
TEST_PREDICTIONS_PATH = RUN_DIR / "final_test_predictions.csv"

np.save(TEST_PROBABILITIES_PATH, test_raw_probabilities)

pd.DataFrame({
    "y_true": y_test,
    "y_pred": y_test_pred,
    "raw_probability": test_raw_probabilities,
    "calibrated_probability": test_probabilities,
}).to_csv(TEST_PREDICTIONS_PATH, index=False)
```

```python
with TEST_RESULTS_PATH.open("w", encoding="utf-8") as f:
    json.dump(test_metrics, f, indent=2)

print("FINAL HELD-OUT TEST RESULTS")
print("=" * 38)
for metric in (
    "accuracy", "precision", "recall", "f1", "fpr", "roc_auc",
    "tn", "fp", "fn", "tp",
):
    print(f"{metric:12s}: {test_metrics[metric]}")

print(f"\nThreshold:          {test_threshold:.6f}")
print(f"Polarity:           {frozen_calibration['polarity']}")
print(f"Inference duration: {test_inference_seconds:.2f} seconds")
print(f"Throughput:         {test_metrics['samples_per_second']:.2f}
samples/sec")
print("\nSaved metrics:", TEST_RESULTS_PATH)
print("Saved predictions:", TEST_PREDICTIONS_PATH)
```

```
FINAL HELD-OUT TEST RESULTS
======================================
accuracy    : 0.719
precision   : 0.9148494288681205
recall      : 0.6473181484202792
f1          : 0.7581755593803786
fpr         : 0.12832550860719874
roc_auc     : 0.7275402763548391
tn          : 557
fp          : 82
fn          : 480
tp          : 881

Threshold:          0.496094
Polarity:           inverted
Inference duration: 10.21 seconds
Throughput:         195.86 samples/sec

Saved metrics: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_controlled_v1\final_test_metrics.json
Saved predictions: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_controlled_v1\final_test_predictions.csv
```

```python
# 5.2 Audit feature leakage and preprocessor inputs

print("Raw input columns passed to preprocessor:")
for column in X_train_raw.columns:
    flags = []
    if column.lower() in {"label", "attack_cat"}:
        flags.append("TARGET / TARGET-DERIVED")
    if column.lower() == "id":
```

```
        flags.append("IDENTIFIER")

    suffix = f"  <-- {', '.join(flags)}" if flags else ""
    print(f"  {column}{suffix}")

print("\nFitted ColumnTransformer configuration:")
for name, transformer, columns in preprocessor.transformers_:
    if name == "remainder" and transformer == "drop":
        continue

    if isinstance(columns, (list, tuple, np.ndarray, pd.Index)):
        column_names = list(columns)
    else:
        column_names = columns

    print(f"\nTransformer: {name}")
    print(f"Type: {type(transformer).__name__}")
    print(f"Columns: {column_names}")

    if isinstance(column_names, list):
        suspicious = [
            col for col in column_names
            if str(col).lower() in {"label", "attack_cat"}
        ]
        if suspicious:
            print("WARNING — potential leakage columns:", suspicious)
```

```
Raw input columns passed to preprocessor:
  id  <-- IDENTIFIER
  dur
  proto
  service
  state
  spkts
  dpkts
  sbytes
  dbytes
  rate
  sttl
  dttl
  sload
  dload
  sloss
  dloss
  sinpkt
  dinpkt
  sjit
  djit
  swin
  stcpb
```

```
  dtcpb
  dwin
  tcprtt
  synack
  ackdat
  smean
  dmean
  trans_depth
  response_body_len
  ct_srv_src
  ct_state_ttl
  ct_dst_ltm
  ct_src_dport_ltm
  ct_dst_sport_ltm
  ct_dst_src_ltm
  is_ftp_login
  ct_ftp_cmd
  ct_flw_http_mthd
  ct_src_ltm
  ct_srv_dst
  is_sm_ips_ports
  attack_cat  <-- TARGET / TARGET-DERIVED

Fitted ColumnTransformer configuration:

Transformer: numeric
Type: Pipeline
Columns: ['dur', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload',
'dload', 'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'stcpb', 'dtcpb',
'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',
'response_body_len', 'ct_state_ttl', 'ct_dst_sport_ltm', 'ct_srv_dst',
'is_ftp_login', 'ct_flw_http_mthd']

Transformer: categorical
Type: Pipeline
Columns: ['proto', 'service', 'state']

# 5.3 Explicitly exclude target-derived and identifier columns

EXCLUDED_COLUMNS = ["label", "attack_cat", "id"]

def prepare_qml_raw_features(frame):
    missing = [col for col in EXCLUDED_COLUMNS if col not in
frame.columns]
    if missing:
        raise ValueError(f"Expected columns missing: {missing}")

    X = frame.drop(columns=EXCLUDED_COLUMNS).copy()

    assert not set(EXCLUDED_COLUMNS).intersection(X.columns)
```

```
    return X

X_train_raw_clean = prepare_qml_raw_features(train_subset)
X_validation_raw_clean = prepare_qml_raw_features(validation_subset)
X_test_raw_clean = prepare_qml_raw_features(test_subset)

print("Clean raw feature counts:")
print("  Train:     ", X_train_raw_clean.shape)
print("  Validation:", X_validation_raw_clean.shape)
print("  Final test:", X_test_raw_clean.shape)

print("\nExcluded:", EXCLUDED_COLUMNS)
print("[OK] No label, attack_cat, or id columns in the raw QML
inputs.")
```

```
Clean raw feature counts:
  Train:      (2000, 42)
  Validation: (1000, 42)
  Final test: (2000, 42)

Excluded: ['label', 'attack_cat', 'id']
[OK] No label, attack_cat, or id columns in the raw QML inputs.
```

```
# 5.4 Inspect preprocessor input schema and fitted feature selection

print("Preprocessor feature-name metadata")

if hasattr(preprocessor, "feature_names_in_"):
    fitted_input_columns = list(preprocessor.feature_names_in_)
    print("Fitted input columns:", len(fitted_input_columns))
    print(fitted_input_columns)
else:
    fitted_input_columns = None
    print("No feature_names_in_ metadata available.")

print("\nConfigured transformer columns:")
for name, transformer, columns in preprocessor.transformers_:
    print(f"\n{name}: {type(transformer).__name__}")
    print("Columns:", columns)

print("\nClean raw feature count:", X_train_raw_clean.shape[1])

if fitted_input_columns is not None:
    excluded_from_clean = sorted(
        set(fitted_input_columns) - set(X_train_raw_clean.columns)
    )
    additional_clean = sorted(
        set(X_train_raw_clean.columns) - set(fitted_input_columns)
    )

    print("Fitted columns absent from clean inputs:",
```

```
excluded_from_clean)
    print("Clean columns absent from fitted schema:",
additional_clean)

Preprocessor feature-name metadata
Fitted input columns: 29
['dur', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload', 'dload',
'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'stcpb', 'dtcpb',
'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',
'response_body_len', 'ct_state_ttl', 'ct_dst_sport_ltm', 'ct_srv_dst',
'is_ftp_login', 'ct_flw_http_mthd', 'proto', 'service', 'state']

Configured transformer columns:

numeric: Pipeline
Columns: ['dur', 'sbytes', 'dbytes', 'rate', 'sttl', 'dttl', 'sload',
'dload', 'sinpkt', 'dinpkt', 'sjit', 'djit', 'swin', 'stcpb', 'dtcpb',
'synack', 'ackdat', 'smean', 'dmean', 'trans_depth',
'response_body_len', 'ct_state_ttl', 'ct_dst_sport_ltm', 'ct_srv_dst',
'is_ftp_login', 'ct_flw_http_mthd']

categorical: Pipeline
Columns: ['proto', 'service', 'state']

Clean raw feature count: 42
Fitted columns absent from clean inputs: []
Clean columns absent from fitted schema: ['ct_dst_ltm',
'ct_dst_src_ltm', 'ct_ftp_cmd', 'ct_src_dport_ltm', 'ct_src_ltm',
'ct_srv_src', 'dloss', 'dpkts', 'dwin', 'is_sm_ips_ports', 'sloss',
'spkts', 'tcprtt']
```

5.5 Clean Preprocessing Pipeline

The controlled experiment excludes the binary target (`label`), target-derived category (`attack_cat`), and record identifier (`id`).

A fresh preprocessing pipeline is fitted exclusively on the selected training subset. Numeric and categorical columns are inferred from the training features, missing values are imputed, categorical values are one-hot encoded, and numeric features are scaled.

The fitted pipeline is then applied unchanged to validation and final-test data. PCA and quantum angle scaling are fitted on the training representation only.

This preprocessing artifact is specific to the controlled QML experiment and does not replace the existing classical pipeline.

```
# 5.5 Fit clean preprocessing on training data only

import numpy as np
import pandas as pd
```

```
import joblib

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# Determine types from the training subset only.
numeric_columns = X_train_raw_clean.select_dtypes(
    include=["number", "bool"]
).columns.tolist()

categorical_columns = [
    col for col in X_train_raw_clean.columns
    if col not in numeric_columns
]

assert set(numeric_columns).isdisjoint(categorical_columns)
assert set(numeric_columns) | set(categorical_columns) == set(
    X_train_raw_clean.columns
)

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore")),
])

clean_preprocessor = ColumnTransformer([
    ("numeric", numeric_pipeline, numeric_columns),
    ("categorical", categorical_pipeline, categorical_columns),
])

# Fit ONLY on the selected training subset.
X_train_clean_transformed = clean_preprocessor.fit_transform(
    X_train_raw_clean
)

X_validation_clean_transformed = clean_preprocessor.transform(
    X_validation_raw_clean
)

X_test_clean_transformed = clean_preprocessor.transform(
    X_test_raw_clean
)
```

```python
def to_dense_float64(matrix):
    if hasattr(matrix, "toarray"):
        matrix = matrix.toarray()
    return np.asarray(matrix, dtype=np.float64)

X_train_clean_transformed =
to_dense_float64(X_train_clean_transformed)
X_validation_clean_transformed = to_dense_float64(
    X_validation_clean_transformed
)
X_test_clean_transformed = to_dense_float64(X_test_clean_transformed)

for name, matrix in [
    ("Train", X_train_clean_transformed),
    ("Validation", X_validation_clean_transformed),
    ("Final test", X_test_clean_transformed),
]:
    assert matrix.ndim == 2
    assert np.isfinite(matrix).all()
    print(f"{name:10s}: {matrix.shape}")

assert (
    X_train_clean_transformed.shape[1]
    == X_validation_clean_transformed.shape[1]
    == X_test_clean_transformed.shape[1]
)

CLEAN_PREPROCESSOR_PATH = (
    RUN_DIR / "qml_clean_preprocessor.joblib"
)
joblib.dump(clean_preprocessor, CLEAN_PREPROCESSOR_PATH)

print("\nNumeric features:", len(numeric_columns))
print("Categorical features:", len(categorical_columns))
print("[OK] Preprocessor fitted on training data only.")
print("Saved:", CLEAN_PREPROCESSOR_PATH)
```

```
Train     : (2000, 132)
Validation: (1000, 132)
Final test: (2000, 132)

Numeric features: 39
Categorical features: 3
[OK] Preprocessor fitted on training data only.
Saved: C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_controlled_v1\
qml_clean_preprocessor.joblib
```

```python
# 5.6 Fit PCA and angle scaling on clean training representation only

from sklearn.decomposition import PCA
```

```
from sklearn.preprocessing import MinMaxScaler
import numpy as np

pca_clean = PCA(n_components=4, random_state=20261012)

X_train_pca_clean = pca_clean.fit_transform(
    X_train_clean_transformed
)
X_validation_pca_clean = pca_clean.transform(
    X_validation_clean_transformed
)
X_test_pca_clean = pca_clean.transform(
    X_test_clean_transformed
)

angle_scaler_clean = MinMaxScaler(feature_range=(0, np.pi))

X_train_angles_clean = angle_scaler_clean.fit_transform(
    X_train_pca_clean
)
X_validation_angles_clean = angle_scaler_clean.transform(
    X_validation_pca_clean
)
X_test_angles_clean = angle_scaler_clean.transform(
    X_test_pca_clean
)

for name, matrix in [
    ("Train angles", X_train_angles_clean),
    ("Validation angles", X_validation_angles_clean),
    ("Final test angles", X_test_angles_clean),
]:
    assert matrix.shape[1] == 4
    assert np.isfinite(matrix).all()
    print(f"{name:18s}: {matrix.shape}")

print("\nExplained variance ratio:",
pca_clean.explained_variance_ratio_)
print(f"Total variance explained:
{pca_clean.explained_variance_ratio_.sum():.4%}")

# Persist the corrected transformations separately.
import joblib

joblib.dump(pca_clean, RUN_DIR / "qml_clean_pca.joblib")
joblib.dump(angle_scaler_clean, RUN_DIR /
"qml_clean_angle_scaler.joblib")

print("\n[OK] PCA and angle scaler fitted on training data only.")
```

```
Train angles      : (2000, 4)
Validation angles : (1000, 4)
Final test angles : (2000, 4)

Explained variance ratio: [0.25035337 0.10293733 0.08749555
0.08013571]
Total variance explained: 52.0922%

[OK] PCA and angle scaler fitted on training data only.
```

5.7 Corrected Experiment Configuration

This experiment uses the clean 42-feature input contract, a training-only fitted preprocessing pipeline, four PCA components, and training-only angle scaling.

The corrected representation differs from the historical run. Results will therefore be reported as a new experiment, not as a direct ablation of the previous circuit.

The data partitions remain fixed. Training, calibration, and final evaluation will follow the same separation rules established in the original protocol.

```python
# 5.7 Freeze corrected experiment configuration

from dataclasses import asdict, dataclass
from pathlib import Path
import json
import numpy as np

@dataclass(frozen=True)
class CleanCircuitCConfig:
    experiment_id: str = "qml_circuit_c_clean_v1"
    training_seed: int = 20261014
    simulator_seed: int = 20261014
    shots: int = 256
    train_samples: int = 2000
    validation_samples: int = 1000
    test_samples: int = 2000
    pca_components: int = 4
    spsa_iterations: int = 20
    spsa_a: float = 0.10
    spsa_c: float = 0.08
    spsa_alpha: float = 0.602
    spsa_gamma: float = 0.101
    max_validation_fpr: float = 0.10
    excluded_columns: tuple = ("label", "attack_cat", "id")

CLEAN_CONFIG = CleanCircuitCConfig()

assert X_train_angles_clean.shape == (
    CLEAN_CONFIG.train_samples, CLEAN_CONFIG.pca_components
```

```
)
assert X_validation_angles_clean.shape == (
    CLEAN_CONFIG.validation_samples, CLEAN_CONFIG.pca_components
)
assert X_test_angles_clean.shape == (
    CLEAN_CONFIG.test_samples, CLEAN_CONFIG.pca_components
)
assert SHOTS == CLEAN_CONFIG.shots

CLEAN_RUN_DIR = QML_ARTIFACT_DIR / CLEAN_CONFIG.experiment_id
CLEAN_RUN_DIR.mkdir(parents=True, exist_ok=False)

clean_config_dict = asdict(CLEAN_CONFIG)
clean_config_dict["excluded_columns"] = list(
    clean_config_dict["excluded_columns"]
)
clean_config_dict["numeric_columns"] = numeric_columns
clean_config_dict["categorical_columns"] = categorical_columns
clean_config_dict["explained_variance_ratio"] = (
    pca_clean.explained_variance_ratio_.tolist()
)
clean_config_dict["explained_variance_total"] = float(
    pca_clean.explained_variance_ratio_.sum()
)

with (CLEAN_RUN_DIR / "experiment_config.json").open(
    "w", encoding="utf-8"
) as f:
    json.dump(clean_config_dict, f, indent=2)

print(json.dumps(clean_config_dict, indent=2))
print("\n[OK] Corrected experiment configuration frozen.")
print("Run directory:", CLEAN_RUN_DIR)

{
  "experiment_id": "qml_circuit_c_clean_v1",
  "training_seed": 20261014,
  "simulator_seed": 20261014,
  "shots": 256,
  "train_samples": 2000,
  "validation_samples": 1000,
  "test_samples": 2000,
  "pca_components": 4,
  "spsa_iterations": 20,
  "spsa_a": 0.1,
  "spsa_c": 0.08,
  "spsa_alpha": 0.602,
  "spsa_gamma": 0.101,
  "max_validation_fpr": 0.1,
  "excluded_columns": [
```

```
    "label",
    "attack_cat",
    "id"
  ],
  "numeric_columns": [
    "dur",
    "spkts",
    "dpkts",
    "sbytes",
    "dbytes",
    "rate",
    "sttl",
    "dttl",
    "sload",
    "dload",
    "sloss",
    "dloss",
    "sinpkt",
    "dinpkt",
    "sjit",
    "djit",
    "swin",
    "stcpb",
    "dtcpb",
    "dwin",
    "tcprtt",
    "synack",
    "ackdat",
    "smean",
    "dmean",
    "trans_depth",
    "response_body_len",
    "ct_srv_src",
    "ct_state_ttl",
    "ct_dst_ltm",
    "ct_src_dport_ltm",
    "ct_dst_sport_ltm",
    "ct_dst_src_ltm",
    "is_ftp_login",
    "ct_ftp_cmd",
    "ct_flw_http_mthd",
    "ct_src_ltm",
    "ct_srv_dst",
    "is_sm_ips_ports"
  ],
  "categorical_columns": [
    "proto",
    "service",
    "state"
```

```
  ],
  "explained_variance_ratio": [
    0.2503533707772313,
    0.10293732772656317,
    0.0874955514420762,
    0.08013571374373046
  ],
  "explained_variance_total": 0.5209219636896012
}

[OK] Corrected experiment configuration frozen.
Run directory: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_clean_v1
```

5.8 Training the Corrected Circuit C

Train the same four-qubit Circuit C architecture using the corrected feature representation and the frozen SPSA configuration.

The training objective uses only the 2,000 training samples. Validation and final-test data are excluded from optimization. Save the final parameters and the complete optimization history in the clean experiment directory.

The previous experiment's parameters and results are not reused.

```python
# 5.8 Train corrected Circuit C with per-iteration checkpoints

import time
import json
import numpy as np
import pandas as pd

CLEAN_CHECKPOINT_PATH = CLEAN_RUN_DIR / "training_checkpoint.npz"
CLEAN_HISTORY_PATH = CLEAN_RUN_DIR / "training_history.csv"
CLEAN_PARAMETERS_PATH = CLEAN_RUN_DIR / "circuit_c_parameters.npy"

assert not CLEAN_CHECKPOINT_PATH.exists()
assert not CLEAN_HISTORY_PATH.exists()
assert not CLEAN_PARAMETERS_PATH.exists()

# Explicitly map the frozen clean configuration to the existing
circuit.
SHOTS_CLEAN = CLEAN_CONFIG.shots
SIMULATOR_SEED_CLEAN = CLEAN_CONFIG.simulator_seed

def predict_probabilities_clean(X_angles, theta, seed_base):
    """Infer probabilities using the clean run's shot and seed
settings."""
    X_angles = np.asarray(X_angles, dtype=np.float64)
    probabilities = np.empty(len(X_angles), dtype=np.float64)
```

```
    for i, angles in enumerate(X_angles):
        probabilities[i] = circuit_c_probability(
            angles,
            theta,
            shots=SHOTS_CLEAN,
            seed_simulator=seed_base + i,
        )

    return probabilities


def clean_training_objective(parameters):
    probabilities = predict_probabilities_clean(
        X_train_angles_clean,
        parameters,
        seed_base=SIMULATOR_SEED_CLEAN,
    )
    return binary_cross_entropy(y_train, probabilities)


def clean_spsa_step(theta, objective, iteration, rng):
    """SPSA update driven only by CLEAN_CONFIG."""
    theta = np.asarray(theta, dtype=np.float64)

    ak = CLEAN_CONFIG.spsa_a / (
        (iteration + 1) ** CLEAN_CONFIG.spsa_alpha
    )
    ck = CLEAN_CONFIG.spsa_c / (
        (iteration + 1) ** CLEAN_CONFIG.spsa_gamma
    )

    delta = rng.choice([-1.0, 1.0], size=theta.shape)

    loss_plus = objective(theta + ck * delta)
    loss_minus = objective(theta - ck * delta)

    gradient = ((loss_plus - loss_minus) / (2.0 * ck)) * delta
    theta_updated = theta - ak * gradient

    if not np.isfinite(theta_updated).all():
        raise FloatingPointError("Non-finite SPSA parameters")

    return theta_updated, {
        "loss_plus": float(loss_plus),
        "loss_minus": float(loss_minus),
        "gradient_norm": float(np.linalg.norm(gradient)),
        "learning_rate": float(ak),
        "perturbation": float(ck),
    }
```

```
clean_rng = np.random.default_rng(CLEAN_CONFIG.training_seed)
theta_clean = clean_rng.uniform(
    -np.pi, np.pi, size=N_TRAINABLE_PARAMS
)

clean_history = []
clean_training_start = time.perf_counter()

for iteration in range(CLEAN_CONFIG.spsa_iterations):
    iteration_start = time.perf_counter()

    theta_clean, diagnostics = clean_spsa_step(
        theta_clean,
        clean_training_objective,
        iteration,
        clean_rng,
    )

    iteration_seconds = time.perf_counter() - iteration_start
    elapsed_seconds = time.perf_counter() - clean_training_start

    clean_history.append({
        "iteration": iteration + 1,
        **diagnostics,
        "iteration_seconds": iteration_seconds,
        "elapsed_seconds": elapsed_seconds,
    })

    np.savez(
        CLEAN_CHECKPOINT_PATH,
        theta=theta_clean,
        completed_iterations=iteration + 1,
        training_seed=CLEAN_CONFIG.training_seed,
        simulator_seed=CLEAN_CONFIG.simulator_seed,
    )
    pd.DataFrame(clean_history).to_csv(
        CLEAN_HISTORY_PATH, index=False
    )

    print(
        f"Iteration {iteration + 1:02d}/"
        f"{CLEAN_CONFIG.spsa_iterations} | "
        f"L+={diagnostics['loss_plus']:.4f} | "
        f"L-={diagnostics['loss_minus']:.4f} | "
        f"|g|={diagnostics['gradient_norm']:.4f} | "
        f"{iteration_seconds:.1f}s"
    )
```

```
clean_training_duration = time.perf_counter() - clean_training_start
np.save(CLEAN_PARAMETERS_PATH, theta_clean)

print(f"\nClean training completed in {clean_training_duration /
60:.2f} minutes.")
print("Checkpoint:", CLEAN_CHECKPOINT_PATH)
print("History:", CLEAN_HISTORY_PATH)
print("Parameters:", CLEAN_PARAMETERS_PATH)
```

```
Iteration 01/20 | L+=1.4416 | L-=1.9737 | |g|=9.4066 | 25.7s
Iteration 02/20 | L+=1.0654 | L-=0.9311 | |g|=2.5465 | 20.8s
Iteration 03/20 | L+=1.0131 | L-=0.8639 | |g|=2.9485 | 20.7s
Iteration 04/20 | L+=0.8096 | L-=0.9643 | |g|=3.1450 | 19.9s
Iteration 05/20 | L+=0.9203 | L-=0.7657 | |g|=3.2166 | 20.0s
Iteration 06/20 | L+=0.7346 | L-=0.8629 | |g|=2.7194 | 20.4s
Iteration 07/20 | L+=0.8184 | L-=0.7112 | |g|=2.3062 | 19.9s
Iteration 08/20 | L+=0.7987 | L-=0.6943 | |g|=2.2783 | 19.8s
Iteration 09/20 | L+=0.6834 | L-=0.7726 | |g|=1.9692 | 20.1s
Iteration 10/20 | L+=0.6700 | L-=0.7486 | |g|=1.7534 | 20.4s
Iteration 11/20 | L+=0.7426 | L-=0.6635 | |g|=1.7816 | 20.2s
Iteration 12/20 | L+=0.7333 | L-=0.6575 | |g|=1.7213 | 20.1s
Iteration 13/20 | L+=0.7165 | L-=0.6518 | |g|=1.4824 | 20.0s
Iteration 14/20 | L+=0.6468 | L-=0.7158 | |g|=1.5917 | 20.5s
Iteration 15/20 | L+=0.6425 | L-=0.7028 | |g|=1.3994 | 19.9s
Iteration 16/20 | L+=0.7029 | L-=0.6395 | |g|=1.4828 | 20.1s
Iteration 17/20 | L+=0.6358 | L-=0.6933 | |g|=1.3544 | 19.9s
Iteration 18/20 | L+=0.6328 | L-=0.6827 | |g|=1.1827 | 20.0s
Iteration 19/20 | L+=0.6843 | L-=0.6307 | |g|=1.2749 | 20.0s
Iteration 20/20 | L+=0.6283 | L-=0.6806 | |g|=1.2502 | 20.0s

Clean training completed in 6.81 minutes.
Checkpoint: C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_clean_v1\
training_checkpoint.npz
History: C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_clean_v1\
training_history.csv
Parameters: C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_clean_v1\
circuit_c_parameters.npy
```

5.9 Validation-Based Calibration — Clean Experiment

Generate predictions on the validation partition using the corrected model and feature representation. Select the output polarity and threshold by maximizing validation F1 subject to a false-positive rate of at most 10%.

The final-test partition remains excluded from this selection process. Calibration metrics and the selected operating point will be saved as experiment artifacts.

```
# 5.9 Validation inference for corrected Circuit C

import time
```

```python
import json
import numpy as np

theta_clean_final = np.load(CLEAN_PARAMETERS_PATH)

assert theta_clean_final.shape == (N_TRAINABLE_PARAMS,)
assert np.isfinite(theta_clean_final).all()

validation_start = time.perf_counter()

validation_probabilities_clean = predict_probabilities_clean(
    X_validation_angles_clean,
    theta_clean_final,
    seed_base=CLEAN_CONFIG.simulator_seed + 100_000,
)

validation_seconds_clean = time.perf_counter() - validation_start

assert validation_probabilities_clean.shape == (len(y_validation),)
assert np.isfinite(validation_probabilities_clean).all()
assert np.all(
    (validation_probabilities_clean >= 0)
    & (validation_probabilities_clean <= 1)
)

CLEAN_VALIDATION_PROBABILITIES_PATH = (
    CLEAN_RUN_DIR / "validation_probabilities.npy"
)
np.save(
    CLEAN_VALIDATION_PROBABILITIES_PATH,
    validation_probabilities_clean,
)

print("[OK] Corrected-model validation inference complete.")
print(f"Samples: {len(y_validation):,}")
print(f"Duration: {validation_seconds_clean:.2f} seconds")
print(f"Mean raw probability:
{validation_probabilities_clean.mean():.4f}")
print(f"Std raw probability:
{validation_probabilities_clean.std():.4f}")
print(
    "Unique probability values:",
    len(np.unique(validation_probabilities_clean)),
)
print("Saved:", CLEAN_VALIDATION_PROBABILITIES_PATH)
```

```
[OK] Corrected-model validation inference complete.
Samples: 1,000
Duration: 9.43 seconds
Mean raw probability: 0.7364
```

```
Std raw probability:  0.1922
Unique probability values: 162
Saved: C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_clean_v1\
validation_probabilities.npy
```

5.10 Threshold Selection — Clean Experiment

Evaluate both raw probability orientations and select the threshold maximizing validation F1 subject to (FPR \leq 0.10).

This selection is performed exclusively on validation labels. If no threshold satisfies the constraint, report that outcome without relaxing the criterion. Freeze the resulting polarity and threshold before final-test evaluation.

```python
# 5.10 Validation-only calibration for corrected Circuit C

import json
import numpy as np
import pandas as pd

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    f1_score,
    precision_score,
    recall_score,
    roc_auc_score,
)


def clean_metrics(y_true, y_pred, probabilities):
    tn, fp, fn, tp = confusion_matrix(
        y_true, y_pred, labels=[0, 1]
    ).ravel()

    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision": float(precision_score(
            y_true, y_pred, zero_division=0
        )),
        "recall": float(recall_score(
            y_true, y_pred, zero_division=0
        )),
        "f1": float(f1_score(y_true, y_pred, zero_division=0)),
        "fpr": float(fp / (fp + tn)) if (fp + tn) else 0.0,
        "roc_auc": float(roc_auc_score(y_true, probabilities)),
        "tn": int(tn),
        "fp": int(fp),
        "fn": int(fn),
```

```
        "tp": int(tp),
    }


def calibrate_clean_threshold(y_true, raw_probabilities, max_fpr):
    candidates = []

    for polarity in ("normal", "inverted"):
        probabilities = (
            raw_probabilities
            if polarity == "normal"
            else 1.0 - raw_probabilities
        )

        thresholds = np.r_[
            np.nextafter(probabilities.max(), np.inf),
            np.unique(probabilities)[::-1],
            np.nextafter(probabilities.min(), -np.inf),
        ]

        for threshold in thresholds:
            predictions = (probabilities >=
threshold).astype(np.int64)
            metrics = clean_metrics(y_true, predictions,
probabilities)

            if metrics["fpr"] <= max_fpr:
                candidates.append({
                    "polarity": polarity,
                    "threshold": float(threshold),
                    **metrics,
                })

    if not candidates:
        raise RuntimeError(
            "No threshold satisfies the predefined validation FPR
constraint."
        )

    # Primary objective: maximum F1.
    # Tie-breakers: precision, recall, then higher threshold.
    candidates.sort(
        key=lambda row: (
            row["f1"],
            row["precision"],
            row["recall"],
            row["threshold"],
        ),
        reverse=True,
    )
```

```
    return candidates[0], pd.DataFrame(candidates)


clean_calibration, clean_feasible_thresholds =
calibrate_clean_threshold(
    y_validation,
    validation_probabilities_clean,
    max_fpr=CLEAN_CONFIG.max_validation_fpr,
)

CLEAN_CALIBRATION_PATH = CLEAN_RUN_DIR / "validation_calibration.json"

with CLEAN_CALIBRATION_PATH.open("w", encoding="utf-8") as f:
    json.dump(clean_calibration, f, indent=2)

print("CLEAN MODEL — VALIDATION CALIBRATION")
print("=" * 42)
print("Selected polarity:", clean_calibration["polarity"])
print(f"Selected threshold: {clean_calibration['threshold']:.6f}")
print(f"Validation FPR:     {clean_calibration['fpr']:.4f}")
print(f"Validation F1:      {clean_calibration['f1']:.4f}")
print(f"Validation ROC-AUC: {clean_calibration['roc_auc']:.4f}")
print(f"Feasible thresholds: {len(clean_feasible_thresholds):,}")

print("\nFull validation metrics:")
for metric in (
    "accuracy", "precision", "recall", "f1", "fpr", "roc_auc",
    "tn", "fp", "fn", "tp",
):
    print(f"  {metric:10s}: {clean_calibration[metric]}")

print("\nSaved calibration:", CLEAN_CALIBRATION_PATH)
```

```
CLEAN MODEL — VALIDATION CALIBRATION
==========================================
Selected polarity: normal
Selected threshold: 0.804688
Validation FPR:     0.0972
Validation F1:      0.7552
Validation ROC-AUC: 0.7263
Feasible thresholds: 91

Full validation metrics:
  accuracy  : 0.72
  precision : 0.9330453563714903
  recall    : 0.6343612334801763
  f1        : 0.7552447552447552
  fpr       : 0.09717868338557993
  roc_auc   : 0.7263129548561722
```

```
  tn        : 288
  fp        : 31
  fn        : 249
  tp        : 432

Saved calibration: C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_clean_v1\validation_calibration.json
```

```python
import json
import time
import numpy as np
import pandas as pd
from sklearn.metrics import confusion_matrix

# Frozen calibration — do not tune using test labels.
frozen_polarity = clean_calibration["polarity"]
frozen_threshold = float(clean_calibration["threshold"])

print("CLEAN MODEL — FINAL TEST EVALUATION")
print("=" * 50)
print(f"Frozen polarity:  {frozen_polarity}")
print(f"Frozen threshold: {frozen_threshold:.6f}")
print(f"Test samples:     {len(y_test)}")

# Generate raw test probabilities.
test_start = time.perf_counter()

test_probabilities_clean = predict_probabilities_clean(
    X_test_angles_clean,
    theta_clean_final,
    seed_base=CLEAN_CONFIG.simulator_seed + 200_000,
)

test_inference_seconds = time.perf_counter() - test_start

# Apply the polarity and threshold selected on validation only.
if frozen_polarity == "normal":
    calibrated_test_probabilities = test_probabilities_clean
elif frozen_polarity == "inverted":
    calibrated_test_probabilities = 1.0 - test_probabilities_clean
else:
    raise ValueError(f"Unexpected polarity: {frozen_polarity}")

test_predictions_clean = (
    calibrated_test_probabilities >= frozen_threshold
).astype(int)

final_test_metrics_clean = clean_metrics(
    y_test,
    test_predictions_clean,
```

```
    calibrated_test_probabilities,
)

final_test_metrics_clean.update({
    "experiment_id": CLEAN_CONFIG.experiment_id,
    "polarity": frozen_polarity,
    "threshold": frozen_threshold,
    "test_samples": int(len(y_test)),
    "inference_seconds": float(test_inference_seconds),
    "samples_per_second": float(
        len(y_test) / test_inference_seconds
    ),
    "shots": int(CLEAN_CONFIG.shots),
    "training_iterations": int(CLEAN_CONFIG.spsa_iterations),
    "training_samples": int(len(X_train_angles_clean)),
    "validation_samples": int(len(X_validation_angles_clean)),
})

np.save(
    CLEAN_RUN_DIR / "test_probabilities.npy",
    test_probabilities_clean,
)

pd.DataFrame({
    "y_true": np.asarray(y_test, dtype=int),
    "y_pred": test_predictions_clean,
    "raw_probability": test_probabilities_clean,
    "calibrated_probability": calibrated_test_probabilities,
}).to_csv(
    CLEAN_RUN_DIR / "final_test_predictions.csv",
    index=False,
)

with open(CLEAN_RUN_DIR / "final_test_metrics.json", "w") as f:
    json.dump(final_test_metrics_clean, f, indent=2)

print("CLEAN MODEL — FINAL TEST METRICS")
print("=" * 45)

for key in [
    "accuracy", "precision", "recall", "f1",
    "fpr", "roc_auc", "tn", "fp", "fn", "tp",
]:
    print(f"  {key:10s}: {final_test_metrics_clean[key]}")

print(f"\nInference time: {test_inference_seconds:.2f} seconds")
print(f"Throughput:     {len(y_test) / test_inference_seconds:.2f}
samples/sec")
print(f"Artifacts:      {CLEAN_RUN_DIR}")
```

```
CLEAN MODEL — FINAL TEST EVALUATION
================================================
Frozen polarity:  normal
Frozen threshold: 0.804688
Test samples:     2000
CLEAN MODEL — FINAL TEST METRICS
============================================
  accuracy  : 0.7025
  precision : 0.9190371991247265
  recall    : 0.6171932402645114
  f1        : 0.7384615384615385
  fpr       : 0.11580594679186229
  roc_auc   : 0.7163689131277171
  tn        : 565
  fp        : 74
  fn        : 521
  tp        : 840

Inference time: 10.43 seconds
Throughput:     191.69 samples/sec
Artifacts:      C:\Projects\Aeges-Q\artifacts\qml\
qml_circuit_c_clean_v1
```

```python
import json
import time
import joblib
import numpy as np
from pathlib import Path
from sklearn.ensemble import RandomForestClassifier

# Reuse the clean QML experiment's existing split and features.
X_classical_train = np.asarray(X_train_angles_clean)
X_classical_val = np.asarray(X_validation_angles_clean)

y_classical_train = np.asarray(y_train, dtype=int)
y_classical_val = np.asarray(y_validation, dtype=int)

assert X_classical_train.shape == (2000, 4)
assert X_classical_val.shape == (1000, 4)
assert len(y_classical_train) == len(X_classical_train)
assert len(y_classical_val) == len(X_classical_val)

classical_rf_config = {
    "experiment_id": "classical_rf_matched_4pca_v1",
    "model": "RandomForestClassifier",
    "n_estimators": 300,
    "random_state": 20261014,
    "features": "same four PCA components scaled to [0, pi] as clean
Circuit C",
    "train_samples": int(len(y_classical_train)),
```

```
    "validation_samples": int(len(y_classical_val)),
    "max_validation_fpr": 0.10,
}

CLASSICAL_RUN_DIR = PROJECT_ROOT / "artifacts" / "qml" /
"classical_rf_matched_4pca_v1"
CLASSICAL_RUN_DIR.mkdir(parents=True, exist_ok=True)

classical_rf = RandomForestClassifier(
    n_estimators=classical_rf_config["n_estimators"],
    random_state=classical_rf_config["random_state"],
    n_jobs=-1,
)

fit_start = time.perf_counter()
classical_rf.fit(X_classical_train, y_classical_train)
classical_fit_seconds = time.perf_counter() - fit_start

# Validation probabilities only. Do not evaluate on the test set yet.
classical_val_probabilities = classical_rf.predict_proba(
    X_classical_val
)[:, 1]

joblib.dump(classical_rf, CLASSICAL_RUN_DIR / "model.joblib")
np.save(
    CLASSICAL_RUN_DIR / "validation_probabilities.npy",
    classical_val_probabilities,
)

with open(CLASSICAL_RUN_DIR / "experiment_config.json", "w") as f:
    json.dump(classical_rf_config, f, indent=2)

print("MATCHED CLASSICAL BASELINE — TRAINED")
print("=" * 48)
print(f"Training samples:       {len(y_classical_train)}")
print(f"Validation samples:     {len(y_classical_val)}")
print(f"Feature dimensions:     {X_classical_train.shape[1]}")
print(f"Training time:          {classical_fit_seconds:.2f} seconds")
print(f"Validation probabilities:
{classical_val_probabilities.shape}")
print(f"Saved to: {CLASSICAL_RUN_DIR}")

MATCHED CLASSICAL BASELINE — TRAINED
================================================
Training samples:       2000
Validation samples:     1000
Feature dimensions:     4
Training time:          1.89 seconds
Validation probabilities: (1000,)
```

```
Saved to: C:\Projects\Aeges-Q\artifacts\qml\
classical_rf_matched_4pca_v1
```

```
classical_calibration, classical_feasible_thresholds = (
    calibrate_clean_threshold(
        y_classical_val,
        classical_val_probabilities,
        max_fpr=0.10,
    )
)

with open(CLASSICAL_RUN_DIR / "validation_calibration.json", "w") as
f:
    json.dump(classical_calibration, f, indent=2)

print("MATCHED CLASSICAL — VALIDATION CALIBRATION")
print("=" * 48)
print(f"Selected polarity:     {classical_calibration['polarity']}")
print(f"Selected threshold:
{classical_calibration['threshold']:.6f}")
print(f"Validation FPR:        {classical_calibration['fpr']:.4f}")
print(f"Validation F1:         {classical_calibration['f1']:.4f}")
print(f"Validation ROC-AUC:
{classical_calibration['roc_auc']:.4f}")
print(f"Feasible thresholds:   {len(classical_feasible_thresholds)}")

print("\nFull validation metrics:")
for key in [
    "accuracy", "precision", "recall", "f1",
    "fpr", "roc_auc", "tn", "fp", "fn", "tp",
]:
    print(f"  {key:10s}: {classical_calibration[key]}")
```

```
MATCHED CLASSICAL — VALIDATION CALIBRATION
================================================
Selected polarity:     normal
Selected threshold:    0.470000
Validation FPR:        0.0972
Validation F1:         0.9046
Validation ROC-AUC:    0.9663
Feasible thresholds:   143

Full validation metrics:
  accuracy  : 0.876
  precision : 0.9499192245557351
  recall    : 0.8634361233480177
  f1        : 0.9046153846153846
  fpr       : 0.09717868338557993
  roc_auc   : 0.9663112976951651
  tn        : 288
```

```
  fp        : 31
  fn        : 93
  tp        : 588

# Matched classical baseline — final held-out test evaluation.
# Uses the same test rows as clean Circuit C.
# Do not tune anything using y_test.

X_classical_test = np.asarray(X_test_angles_clean)
y_classical_test = np.asarray(y_test, dtype=int)

assert X_classical_test.shape == (2000, 4)
assert len(y_classical_test) == len(X_classical_test)

frozen_classical_polarity = classical_calibration["polarity"]
frozen_classical_threshold = float(classical_calibration["threshold"])

test_start = time.perf_counter()

classical_test_probabilities = classical_rf.predict_proba(
    X_classical_test
)[:, 1]

classical_test_inference_seconds = time.perf_counter() - test_start

if frozen_classical_polarity == "normal":
    classical_test_calibrated_probabilities =
classical_test_probabilities
elif frozen_classical_polarity == "inverted":
    classical_test_calibrated_probabilities = (
        1.0 - classical_test_probabilities
    )
else:
    raise ValueError(
        f"Unexpected polarity: {frozen_classical_polarity}"
    )

classical_test_predictions = (
    classical_test_calibrated_probabilities
    >= frozen_classical_threshold
).astype(int)

classical_final_test_metrics = clean_metrics(
    y_classical_test,
    classical_test_predictions,
    classical_test_calibrated_probabilities,
)

classical_final_test_metrics.update({
    "experiment_id": classical_rf_config["experiment_id"],
```

```
    "polarity": frozen_classical_polarity,
    "threshold": frozen_classical_threshold,
    "test_samples": int(len(y_classical_test)),
    "training_seconds": float(classical_fit_seconds),
    "inference_seconds": float(classical_test_inference_seconds),
    "samples_per_second": float(
        len(y_classical_test) / classical_test_inference_seconds
    ),
    "n_estimators": int(classical_rf_config["n_estimators"]),
    "training_samples": int(len(y_classical_train)),
    "validation_samples": int(len(y_classical_val)),
    "feature_dimensions": int(X_classical_train.shape[1]),
})

np.save(
    CLASSICAL_RUN_DIR / "test_probabilities.npy",
    classical_test_probabilities,
)

pd.DataFrame({
    "y_true": y_classical_test,
    "y_pred": classical_test_predictions,
    "raw_probability": classical_test_probabilities,
    "calibrated_probability": classical_test_calibrated_probabilities,
}).to_csv(
    CLASSICAL_RUN_DIR / "final_test_predictions.csv",
    index=False,
)

with open(CLASSICAL_RUN_DIR / "final_test_metrics.json", "w") as f:
    json.dump(classical_final_test_metrics, f, indent=2)

print("MATCHED CLASSICAL — FINAL TEST METRICS")
print("=" * 48)
print(f"Frozen polarity:  {frozen_classical_polarity}")
print(f"Frozen threshold: {frozen_classical_threshold:.6f}")

for key in [
    "accuracy", "precision", "recall", "f1",
    "fpr", "roc_auc", "tn", "fp", "fn", "tp",
]:
    print(f"  {key:10s}: {classical_final_test_metrics[key]}")

print(f"\nTraining time:  {classical_fit_seconds:.2f} seconds")
print(f"Inference time: {classical_test_inference_seconds:.4f}
seconds")
print(f"Throughput:     {len(y_classical_test) /
classical_test_inference_seconds:.2f} samples/sec")
print(f"Artifacts:      {CLASSICAL_RUN_DIR}")
```

```
MATCHED CLASSICAL — FINAL TEST METRICS
================================================
Frozen polarity:  normal
Frozen threshold: 0.470000
  accuracy  : 0.8525
  precision : 0.9412251655629139
  recall    : 0.8354151359294636
  f1        : 0.8851693265862203
  fpr       : 0.1111111111111111
  roc_auc   : 0.9514205816168954
  tn        : 568
  fp        : 71
  fn        : 224
  tp        : 1137

Training time:  1.89 seconds
Inference time: 0.2936 seconds
Throughput:     6810.84 samples/sec
Artifacts:      C:\Projects\Aeges-Q\artifacts\qml\
classical_rf_matched_4pca_v1
```

```python
# Final side-by-side comparison artifact

comparison_metrics = [
    "accuracy",
    "precision",
    "recall",
    "f1",
    "fpr",
    "roc_auc",
    "tn",
    "fp",
    "fn",
    "tp",
]

comparison_rows = []

for model_name, metrics in [
    ("Quantum Circuit C", final_test_metrics_clean),
    ("Matched Random Forest", classical_final_test_metrics),
]:
    row = {"model": model_name}
    row.update({metric: metrics[metric] for metric in
comparison_metrics})
    comparison_rows.append(row)

comparison_df = pd.DataFrame(comparison_rows)

comparison_df.to_csv(
```

```
    CLEAN_RUN_DIR / "matched_model_comparison.csv",
    index=False,
)

comparison_df.to_csv(
    CLASSICAL_RUN_DIR / "matched_model_comparison.csv",
    index=False,
)

comparison_metadata = {
    "protocol": "Matched train/validation/test partitions",
    "training_samples": 2000,
    "validation_samples": 1000,
    "test_samples": 2000,
    "feature_representation": "Four PCA components, scaled to [0,
pi]",
    "threshold_selection": "Validation only, maximize F1 subject to
FPR <= 0.10",
    "test_thresholds": {
        "quantum_circuit_c": {
            "polarity": frozen_polarity,
            "threshold": float(frozen_threshold),
        },
        "matched_random_forest": {
            "polarity": frozen_classical_polarity,
            "threshold": float(frozen_classical_threshold),
        },
    },
    "caveat": (
        "Simulator timings are environment-specific. "
        "No quantum advantage is established."
    ),
}

with open(CLEAN_RUN_DIR / "matched_comparison_metadata.json", "w") as
f:
    json.dump(comparison_metadata, f, indent=2)

print(comparison_df.round(4).to_string(index=False))
print(f"\nSaved comparison artifacts to:\n  {CLEAN_RUN_DIR}")
```

```
                model  accuracy  precision  recall     f1    fpr
roc_auc  tn  fp  fn   tp
    Quantum Circuit C    0.7025     0.9190  0.6172 0.7385 0.1158
0.7164 565  74 521  840
Matched Random Forest    0.8525     0.9412  0.8354 0.8852 0.1111
0.9514 568  71 224 1137

Saved comparison artifacts to:
  C:\Projects\Aeges-Q\artifacts\qml\qml_circuit_c_clean_v1
```